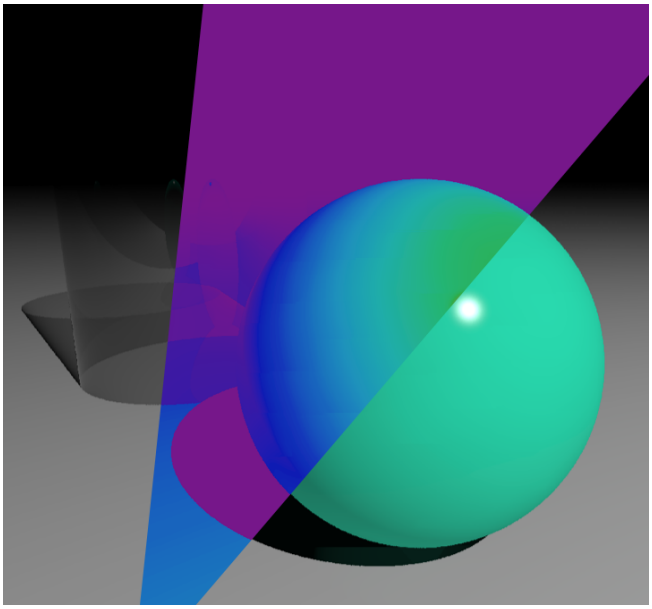
Introduction to Radiance



Guillaume Le Gall

<guillaume.le-gall@univ-smb.fr>

ESBC - ENER858 Advanced Radiation Modelling in Complex Media
Solar Academy
Université Savoie Mont Blanc

17th April 2026

Contents

1 The Radiance rendering engine

1.1 About Radiance

RADIANCE [10] is a sophisticated and validated rendering engine for lighting visualisation and calculation, initially developed by Greg Ward at the Lawrence Berkeley National Laboratory (LBNL), Berkeley, California, US. It combines advanced Monte Carlo backwards ray-tracing with optimisation procedures to efficiently solve the rendering equation [4] by considering the specular and diffuse reflections and transmissions at any level of a given three-dimensional lit scene. It can handle complex geometries and fenestration systems, as well as a variety of materials. Its global illumination model evaluates how light interacts between the various surfaces in the scene, taking into account the diffuse intrerreflections between objects, while at the local scale, the software allows for an accurate description of radiative processes (emission, reflection, transmission and scattering). Thus, RADIANCE does not focus on the rendering of photorealistic scenes solely but produces results which are physically correct and accurately transcribe reality, as measured by a physical optical sensor. Over the last two decades, it has consequently been adopted by

researchers, but also architects, lighting designers or anyone eager to produce visually and physically accurate lighting renderings. Nowadays, a significant part of lighting simulation software relies on RADIANCE (*e.g.* Honeybee [5], ClimateStudio [9]).

1.2 A UNIX-based environment

RADIANCE is an open-source set of over 100 individual programs working in a UNIX environment.

UNIX is a family of computer operating systems based on sets of simple tools, or programs, each of them being dedicated to a specific and rather simple task. A command-line interface (shell) is used to communicate with the operating system and compute complex workflows by combining these initial tools. This means that no GUI (Graphical User Interface) is available, which intriscally allows for a great flexibility and almost illimited freedom in what can be done, but at the price of a steep learning curve.

Programs are thus launched by typing commands in the shell. As a rule of thumb, each command, written in a line-by-line fashion, takes an input `in_` and returns an output `_out` (Fig. 1). If any, error messages are also output by the command (`_err`).

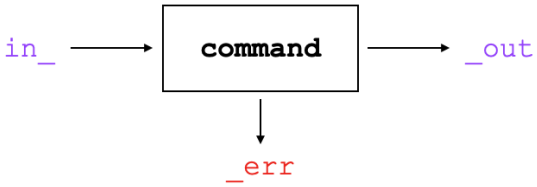


Figure 1: Standard pipeline for a UNIX command.

An introduction to the fundamentals of UNIX environments and the main useful commands within the context of this course are provided in section (2.3).

2 Installation

This section provides guidance for the the setup of the UNIX environment (2.1) and the installation of RADIANCE (2.2) on your personnal computer. The assumption is made here that your machine is currently running under the Windows operating system (version 10 or higher). If you are currently using MacOs, the installation of RADIANCE may slightly defer and you are advised to look for the relevant installation guides on the Internet. If you already have Linux installed, you can skip subsection 2.1.

2.1 Setting-up the UNIX environment

Within the scope of this workshop, we will be using Ubuntu [1] as our UNIX operating system. Ubuntu is probably the most commonly used OS on GNU/Linux and is freely available for download. Hold on, did you say Linux? In fact, Ubuntu is

made to work under Linux and is thus not immediately available on Windows-based machines. Conversely to GNU/Linux and MacOS, Windows is not built on top of a UNIX-like OS. Since a UNIX environment is required for RADIANCE to work, this means that we will have to install it ourselves. The step-by-step installation is described hereinafter.

Step 1 Go to "Update & Security" in the settings of your Windows machine.

Step 2 Click on the "For developers" section in the left-hand panel and select the "Developer mode" option.

Step 3 Launch the "Control Panel" and go to the "Programs" section.

Step 4 Right below "Programs and Features", you should notice a "Turn Windows features on or off" subsection. Click on it and enbale the "Windows Subsystem for Linux" option.

Step 5 Click "OK" and reboot your system by restarting your computer.

Step 6 Search for "bash" in the research bar. You should now be able to select the BASH shell, which is the GNU/Linux command interpreter. Click on it and a window as in Figure 2 should appear.

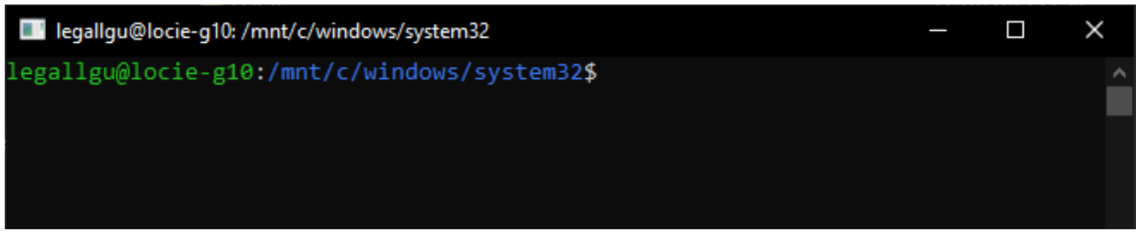


Figure 2: Screenshot of the BASH shell.

If no GNU/Linux operating system has been installed on your computer yet, you should receive a message like that:

```
"Windows Subsystem for Linux has no installed distributions.  Distribu
tions can be installed by visiting the Microsoft Store:  https://aka.m
s/wslstore
Press any key to continue..."
```

Step 7 To install Ubuntu, either type "`lxrun/install`" in the BASH shell and press "Enter" or download it direclty from the Microsoft Store app.

Step 8 Create a UNIX username and password. Though not compulsory, it is advised that these match with your Windows login information.

Step 9 Close the BASH shell and launch it again the same way as at Step 6.

To check if everything is working properly, type "`ls`" in the command intepreter and press "Enter". The shell should now display a list of all the elements contained in the current working directory, whose path is specified by the blue strings in the command line, namely `/mnt/c/windows/system32` in my case (Fig. 2).

2.2 Installation of Radiance

Now we have setup our UNIX environment, we are ready to install RADIANCE. Its installation is quite straightforward.

In that aim, visit [10], and go to the "Download/Install" section. Click on "Radiance Installers", which should redirect you towards a GitHub hyperlink. Once on GitHub, select the latest release, *i.e.* RADIANCE 5.4a at the time this document is written. Download the executable file for Windows (and not Linux!), which should be named something similar to "Radiance_4a16c6c4_Windows.exe". Run the downloaded file and follow the installation steps. Make sure here to **tick the box setting the Radiance path for all users**. Once done, click on "Finish". and you now have RADIANCE installed on your computer!

Before diving into it, let's check if everything has been setup correctly. Go to your C: directory. You should now be able to see a new folder named "Radiance". Click on it. Two subfolders "bin" and "lib" should appear. The "bin" folder contains all the UNIX programs specific to RADIANCE, while "lib" gathered other useful files which will be used by default by the software. It is worth noting here that some programs are missing, with respect to the GNU/Linux or MacOs installations. In fact, because RADIANCE is a UNIX-based tool, some commands calling external dedicated UNIX programs cannot be implemented for Windows. For instance, the X11 visualiser (X Window System) often used with RADIANCE will not be avaiable here. Another quick and easy way to ensure the correct installation of the software is to open the dedicated bash shell and enter a RADIANCE command, *e.g.* `oconv`. If the following lines are in turn displayed, the installation is successful:

```
#?RADIANCE
oconv
FORMAT=Radiance_octree
```

Erratum: For people using a computer from the university, look for 'radiance.bat' in the Windows search bar and click on it to launch the BASH shell specific to RADIANCE. You may also have at this point to set the direction paths for the computer to find the directories where RADIANCE programs and resources are located. In that aim, please enter the two following lines prior to any other commands (only once you are located in the work directory):

```
$ set PATH=.;C:\openstudio-{version}\Radiance\bin;$PATH
$ set RAYPATH=.;C:\openstudio-{version}\Radiance\lib;$RAYPATH
```

with *e.g.* `version` = `2-9-1`.

2.3 An introduction to the UNIX environment for Radiance

Both Ubuntu and RADIANCE are now installed on our computer. Before creating our first gorgeous renderings, let's review the UNIX fundamentals that will be useful within the context of a RADIANCE utilisation from the BASH shell.

The most useful UNIX commands are listed in Table 1. They can be call as there are, or, if available, with additional options using a hyphen:

```
$ command -option
```

The list of available options for a given command can be accessed with the `man` program (Tab. 1). The output of a command can be written in an external file, displayed on the screen or taken as an input by another command. To redirect `_out` into a file, use the following syntax:

Table 1: Useful UNIX commands within the scope of a RADIANCE utilisation.

Cmd	Use	Description	Comments
`cd`	`cd _path`	Move from the current directory to a new directory of path `_path`	Path are case- and space-sensitive and should be specified as follow: "Users/Username/../"
`cd ..`	`cd ..`	Move to the immediate previous directory	
`pwd`	`pwd`	Figure out the present working directory	
`cp`	`cp _file _new_path`	Copy a file	
`man`	`man _command`	Access the manual page of a given command	
`ps`	`ps`	Display the processes which are currently running	Use `-x` to show the background processes as well
`ls`	`ls`	List all the elements in the working directory	Use `-l` to obtain more detailed information
`mv`	`mv _current_path _new_path`	Move a file	
`cat`	`cat _file_1 _file_2`	Concatenate files and print to standard output	
`wc`	`wc _file`	Evaluate the number of lines, words and bytes in a file	
`echo`	`echo` "text"	Display a line of text	
`grep`	`grep` "text"	Print lines matching a pattern of text	
`kill`	`kill PID`	Kill a running process	PID is the process ID, which can be accessed using `ls`

```
$ command > file.ext
```

If >> is used instead, the output will be appended at the end of the file. Conversely, to take the content of a file and use it as an input for a given command, use <:

```
$ command < file.ext
```

Sometimes, the output of a first command needs to be taken as an input for a second command. This process is referred to as "piping" and called using the | operator:

```
$ command_1 | command_2
```

If `command_1` generates here an image file and `command_2` calls a visualiser, this command line would allow for both the creation and display of the image. Redirection and piping are efficient methods to transfer data between different files and programs, and are quite common in RADIANCE.

Two other useful characters are `?` and `*`, which are called to find strings with a single or a multiple missing characters, respectively, in a given file or directory. For instance, the command `$ ls chi*` will return all the elements in the current directory beginning with "chi", (*e.g.* "chichi", "chill", "chicken", "chips") while `$ ls chi*ken` will output all the elements containing the string of character "chi_ken" (*i.e.* only "chicken" here). Now, try it by yourself and find all the elements in the "Radiance\bin" folder starting by "gen". You should obtain something like that:

```
genBSDF.exe     genblinds.exe  genclock.exe    gendaymtx.exe
genprism.exe    genrhgrid.exe  genskyvec.exe   genworm.exe
genambpos.exe   genbox.exe     gendaylit.exe   genklemsamp.exe
genrev.exe      gensky.exe     gensurf.exe
```

The RADIANCE `gen` commands correspond to "generators", but we will come back to that later on. Then, try to figure out which RADIANCE command is hidden behind "o_onv.exe". You should obtain here a single string output.

Finally, more complex scripting, *e.g.* iteration loops, is possible directly *via* the BASH shell. This will yet not be addessed here as it falls outside the scope of this course.

More information about UNIX commands for RADIANCE are available at [2].

3 R1 - A first simple rendering

We will first render a simple scene containing four basic 3D objects with various geometries and materials. The objective here is to become familiar with the RADIANCE environment and its different commands.

3.1 General information

The rendering process in RADIANCE can be split into four stages:

1 Scene geometry definition

2 Surfaces materials definition

3 Lighting simulation and rendering

4 Image manipulation and analysis

The main programs creating the link between these different stages within the global workflow are described in Figure 3."

In this exercise, we will focus on stages **1** to **3**. The different elements for a scene are stored in text files before being compiled using `oconv`. There is not any fixed rule here, but the general agreement is to put geometries, materials and light sources in separate files. The ".rad" extension is used for geometries and light sources while ".mat" is employed for materials files. A dedicated file also exists for the sky itself, but this will be addressed later on.

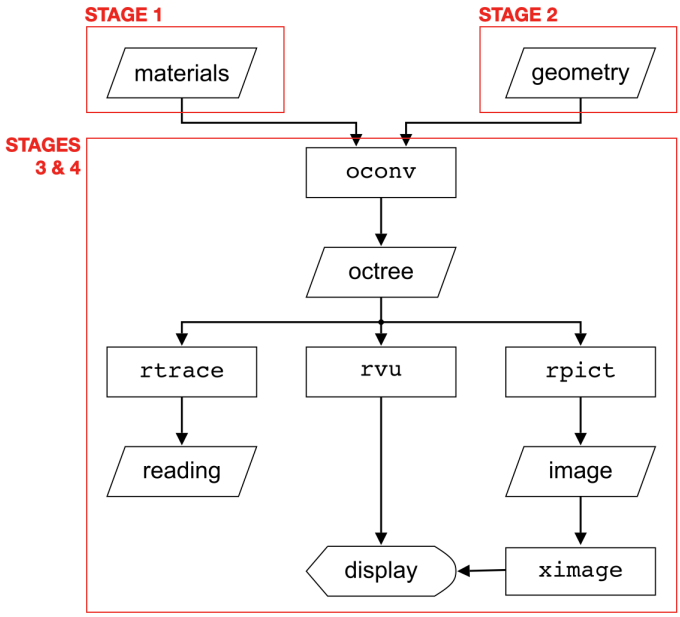


Figure 3: Main programs involved in the Radiance rendering system [3].

3.2 Geometries and materials definition

In RADIANCE, objects are defined using "primitives", which are divided into four distinct types (`type`): geometries, materials, textures and patterns. A list of the main primitives is provided in Appendix A. Any primitive follows the same four-line syntax:

```
modifier type identifier
i str1 str2 ...  stri        strings
0                            integers
j ft1 ft2 ...  ftj           floats
```

The identifier (`identifier`) corresponds to the ID of the defined primitive. You can choose any name as long as it is space-free and unique within your project. The modifier (`modifier`) is used to chain primitives together (Fig. 4). For instance,

to apply a material of identifier `blue_plastic` to a given geometry, simply use `blue_plastic` as the modifier for your geometry. Each object must be assigned at least a geometry and a material. Patterns (alteration of the surface normal) and textures (changes in the material reflectance) are optional but can be used to create complex and realistic material. The modifying primitive must always be defined prior to the modified primitive. If no modifier has to be applied, `void` must be employed.

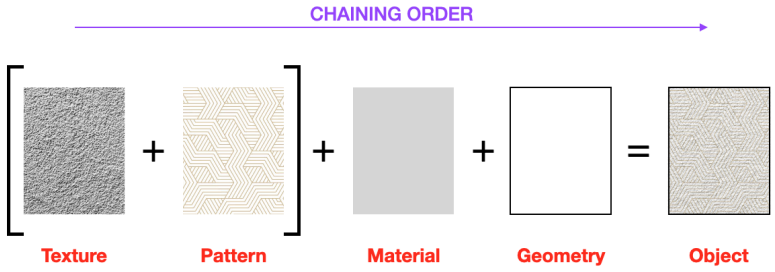


Figure 4: Primitives chaining process.

3.2.1 Describing the geometry

There are basically two ways to create a 3D RADIANCE geometry. One can either use a dedicated CAD software to build the geometry and export it as a text file with a compatible format, or conceive it from scratch by mixing RADIANCE primitives manually. Within the scope of this exercise, we will stick to the latter method, which would be cumbersome and less convenient for complex geometries, but remains rather handy for simple shapes like here.

For instance, the following code will create a flat blue plastic sphere with centre coordinates (2, 0.7, 1) and a radius of 3.5 (comments are displayed with the "#" character):

```
# Material - Flat blue plastic
void plastic flat_blue_plastic
0
0
5 0 0 1 0 0

# Geometry - Sphere
flat_blue_plastic sphere my_sphere
0
0
4 2 .7 1 3.5
```

RADIANCE also provides useful programs, called generators, to create specific geometries. The following command line produces a box with a 4×5 base and 2.5 height:

```
$ genbox flat_blue_plastic my_box 4 5 2.5
```

The complete list of generators alongside all RADIANCE commands and how to use them is detailed at [8]. Commands can either be launch from the shell directly, or stored in a text file to be launched when the file is called. In the latter case, a "!" must precede the command name in the text file:

```
$ !genbox flat_blue_plastic my_box 4 5 2.5
```

You may have noticed that quantities are unit-free, which is always the case in RADIANCE. The coordinate system is also specific to the software, with the x- and y-axes facing East and North, respectively, and the z-axis pointing towards the zenith (Fig. 5).

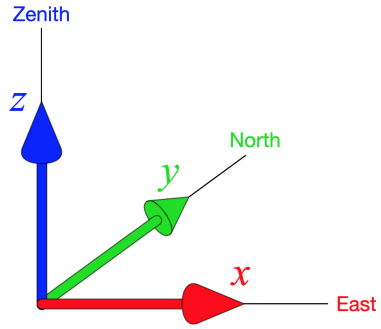


Figure 5: RADIANCE coordinate system.

Two additional useful commands are `getbbox`, which returns the spatial boundaries of a given object in the 3D space, and `xform`, which allows for the translation and/or rotation of one or multiple objects in space.

3.2.2 Describing the material

There is a total of 25 different materials and 12 additional modifiers available in RADIANCE, which let us plenty of room to build sophisticated objects. What differentiates the materials from one to another is their light-matter interaction properties, describing how they disturb the incident light on the object (*e.g.* reflectance, specularity, roughness). `plastic` remains the most employed material for non-emitting surfaces (*e.g.* wood, paper, concrete, plastic, fabric). The example of `flat_blue_plastic` hereinabove gives an example of how it would be defined in context:

```
# Material - Flat blue plastic
void plastic flat_blue_plastic
0
0
5 0     0     1     0     0
  ref_R ref_G ref_B spec  rough
```

All values must range between 0 and 1. Textures and patterns are used to described more realistic and detailed surfaces. As for light sources, the available materials are `light`, `illum`, `glow` and `spotlight`.

The main materials, textures and patterns are listed in Appendix A and the complete list is available at [6]. Some examples of simple material and light source definitions are also presented at [7].

3.3 Visualisation of the scene

Once the geometries, materials and light sources are defined in their respective text files, the next step is to compile the scene into an octree with `oconv`. The position of the different files matters here, as materials have to be called before geometries and light sources.

```
$ oconv materials.mat geometries.rad light_sources.rad > my_scene.oct
```

Here, the output has been stored in an octree file (`my_scene.oct`). To display the result on screen in an interactive window for a given view defined by `view.vf`, the following command can be used:

```
$ rvu -vf view.vf my_scene.oct
```

You can try to change the viewpoint and view direction from the interactive window, or by changing the values in the view file directly. Finally, to generate a still high-resolution image of your scene, call the `rpict` command:

```
$ rpict -vf view.vf my_scene.oct > my_scene_rpict.hdr
```

This process takes some time, so make sure everything is properly settled before calling `rpict`. For instance, adjust the exposure within the image if the scene is under/overexposed. This can be done *via* the interactive window provided by `rvu` by typing *e.g.* "`exposure +1`" in the command bar. You may want at this stage to run the program in the background. To do so, add "`&`" at the end of your command line. The produced image is in HDR format, so you may not be able to dislay it on your screen using a standard visualiser. Since the X Window System is not available on Windows systems, a solution is to convert your HDR image into another readable format (*e.g.* TIFF) using the `ra_tiff` command.

You can also correct the exposure in the scene after calling `rpict`, by applying a powerful RADIANCE tool for antialising and exposure adjustment called `pfilt` to your HDR image directly. For instance, the following command reduces the exposure in the scene by 0.5 f-stops (`-e`) and stores the corrected image in "my_scene_rpict_pfilt.hdr".

```
$ pfilt -e 0.5 my_scene_rpict.hdr > my_scene_rpict_pfilt.hdr
```

Another tool that may be useful at the design stage is `objview`, which displays a geometry in an interactive window without requiring a compilation. This allow you to preview your object without even defining a material or a light source. To visualise your geometry with `objview`, use the following command:

```
$ objview geometries.rad
```

3.4 Your move!

You will now generate your first renderings. At the end of the session, please make sure to send me all your **Radiance files and renderings**, together with you **summary report**. You can use here the internal Filesender program from the university and send everything to my email address.

1. Start by creating a 4×3 rectangular surface using the `polygon` primitive and assign it the material `light_grey` present in the file "materials.mat", as shown in Figure 6. Store your object in a file named "room.rad".

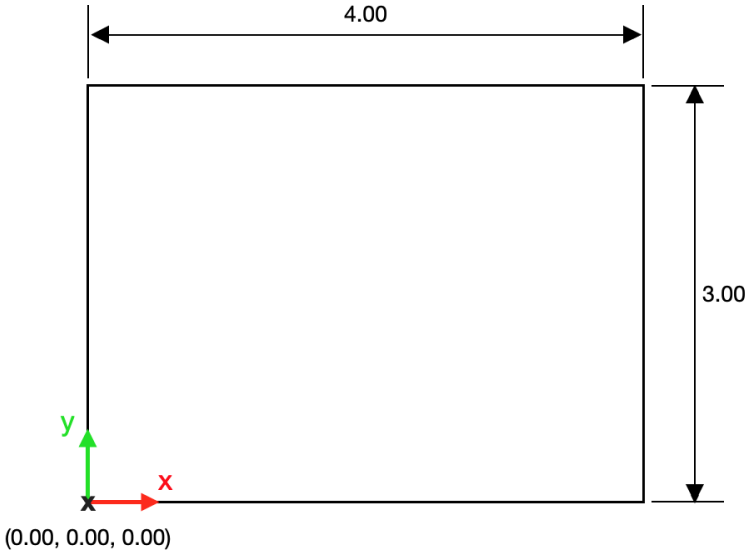


Figure 6: Schematic of the rectangular surface in the RADIANCE coordinate system.

2. Now create a first simple 3D object (*e.g.* sphere, cone, cylinder) of material `plastic` with a glossy surface. You are free to choose the colour and size you want, but your object should fit in a 0.5 width cube. What parameter did you act on to make the surface glossy? Store it in a dedicated ".rad" file.

3. Create a second object with a different geometry. Apply it a a different colour than previously and a `metal` material. It can also be of different size, but its dimensions must not exceed 0.5 in each direction. Compare its material properties with your first object and interprete the differences.

4. Your third object must have a `glass` material with a total transmittance $T_n = 0.62$. RADIANCE `glass` materials take as input the transmittivity t_n, which can be obtained from T_n *via* the following equation:

$$t_n = \frac{\sqrt{0.8402528435 + 0.0072522239T_n^2} - 0.9166530661}{0.0036261119T_n}$$

Here again, make sure that it can be contained within a 0.5 width box.

5. For your final object, you will mix its material with a texture and a pattern to obtain a dusty wooden surface. In that aim, chain the relevant texture and pattern in the "materials.mat" file with the "`light_wood`" material and apply it to a geometry of your choice with similar dimensions than previous objects.

6. Using `xform`, arrange your objects on the squared surface so that their respective centres are located at points A, B, C and D, as shown in Figure 7. A convenient way to procede can be to create a dedicated file (*e.g.* "my_objects.rad") to store these commands. What are the dimensions of the total place occupied by all your objects as returned by `getbbox`?

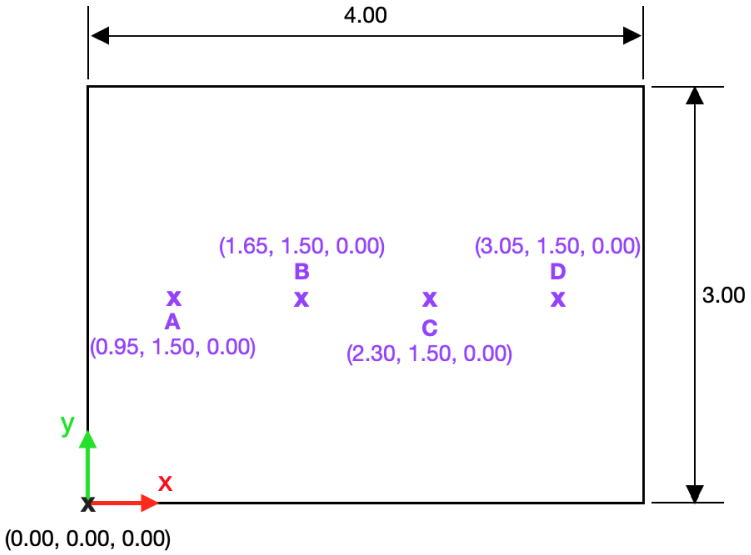


Figure 7: Location of the centres for the four objects on the rectangular surface.

7. Now, use `oconv` to compile your geometries, materials and light sources. Store the result in a dedicated file named "scene.oct". Use `rvu` to visualise the scene and make adjustements if required. Employ here the view "inclined.vf" in the "views" folder.

8. When you are happy with your rendering, generate both a HDR and TIFF image of it using the `rpict` and `ra_tiff` commands, respectively. Use here the view file "visualisation.vf" and following parameters for `rpict`:

```
-x 2048 -y 1468 -ab 2 -aa .15 -ar 128 -ad 512 -as 256
```

4 R2 - Simulate and analyse lighting within an indoor space

In this section, we will simulate the incoming daylight within a simple room and analyse its provision in terms of luminance and illuminance levels.

4.1 Simulating daylight

Light is generally either produced by the sun (natural light) or by electric lamps (artificial light). So far, we have been using a single standard electric light bulb defined with the `light` material to lit our scene. Other more specific materials can be employed depending on the nature of the light source. The four main ones (`light`, `illum`, `glow` and `spotlight`) are listed in Table 2. The goal here is yet to illuminate the scene with natural light, which is always preferred (if properly managed!) to artificial lighting to lit indoor environments, due to its proven benefits on the human vision, health and productivity.

Because daylight is intrinsically variable, a specific RADIANCE command called `gensky` is used to simulate its contribution to the scene illumination. `gensky` defines the distribution of radiance for the sky hemisphere and ground, as well as the material and geometry for the sun. The materials and geometries for the sky and ground should be manually specified by the user. As an example, the following command creates a sunny sky with a source description of the sun (`+s`) for April 8th at 13h15 in Paris (latitude (`-a`): 48.9 °; longitude (`-o`): 2.3 °), and outputs the result in the file "sky.mat":

```
$ gensky 4 8 13:15 +s -a 48.9 -o 2.3 -m -15 > sky.mat
```

To generate a CIE overcast sky instead, `-c` must be added to the hereinabove command. In this case, there is no need to specify any latitude or longitude, as it does not depend on the site location. Again, the complete list of available options can be accessed from [8]. When creating the octree scene with `oconv`, the file corresponding to the sky contribution "sky.mat" is simply call as a standard artificial lamp. Make sure here to also call the related user-defined materials and geometries, usually stored in a single dedicated file "outside.rad".

```
$ oconv materials.mat geometries.rad sky.mat outside.rad >
                        my_scene.oct
```

4.2 Analysing the scene

RADIANCE provides multiple visual and numerical tools to analyse the rendered scenes at a post-processing stage. In this section, we will learn how to produce false colour maps, as well as visualise luminance and illuminance levels within a given scene.

False colour mapping is a powerful visual technique to ease the interpretation of a lighting rendering, as the human eye is better at distinguishing colours than shades of grey. To generate an image in false colours of a rendered scene, the `falsecolor` command is used. As an example, we will consider here the rendering of a simple sphere, whose luminance image as produced by `rpict` is stored in "sphere.hdr" (Fig. 8).
The following command line produces a false colour luminance mapping of the scene from the image "sphere.hdr" and outputs the result in "sphere_fc.hdr" (Fig. 9.a).

```
$ falsecolor -ip sphere.hdr -n 10 -s 250 > sphere_fc.hdr
```

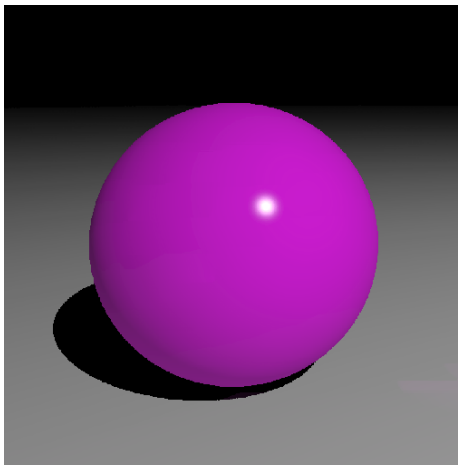


Figure 8: Rendering of a simple magenta sphere with a glossy surface as produced by `rpict`. The image is stored in "sphere.hdr".

The luminance scale is discretised here into 10 different colours (-n) and majorised by 250 cd.m^{-2} (-s). It comes under the user authority to adjust these values accordingly to the range of luminance present in the scene.

It is also possible to compute only the luminance or illuminance colour lines on top of the rendered HDR image. In that case, the `-cl` option must be added when calling `falsecolor`. The example below overlays luminance colour lines onto the image of the sphere "sphere.hdr" (Fig. 9.b).

```
$ falsecolor -ip sphere.hdr -cl -n 10 -s 250 > sphere_fc_cl.hdr
```

To compute the illuminance contour lines instead, the command should be slightly updated:

```
$ falsecolor -i sphere_i.hdr -p sphere.hdr -cl -n 10 -s 2000 -l lux >
                              sphere_fc_cl_i.hdr
```

The unit of the false colour scale has been changed to lux here (-l) and its maximum value increased to 2000 lux. Please note here that the input "sphere_i.hdr" is no more a luminance but an illuminance image, generated by calling `rpict` with the `-i` option.

In both cases, care must be taken if you have adjusted the exposure in your rendered image, *via* either the interactive window or `pfilt`. Make sure to use the raw and not the exposure-corrected image to compute the false colour maps and contour lines.

4.3 Your move!

The goal of the exercise here is to simulate the daylight provision in a simple room with one opening and analyse the rendered images at a post-processing stage. Similarly to the last sesssion, please make sure to send me all your **Radiance files and**

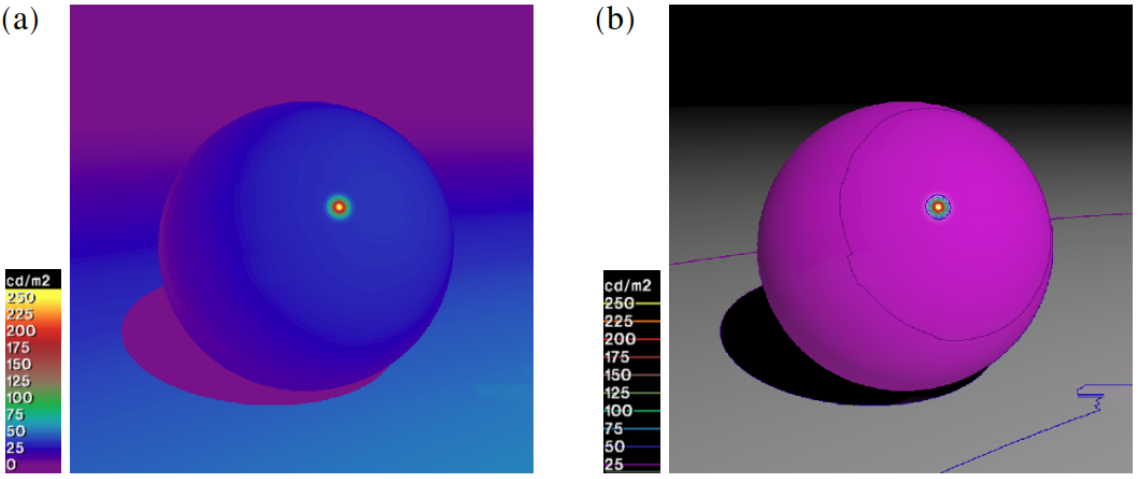


Figure 9: False colour luminance mapping (a) and contour lines (b) of the simple magenta sphere. Images are stored in "sphere_fc.hdr" and "sphere_fc_cl.hdr", respectively.

renderings, together with you **summary report** at the end of the workshop.

1. Start by creating a rectangular box of lentgh 4, width 3 and height 2.9 using the `genbox` command, as shown in Figure 10. The south façade must have a 3×1.5 opening to welcome the window glazing. A tip to create a hole in a wall is to convert the corresponding rectangular face produced by `genbox` in a polygon with coincident edges to give the appearance of an opening. Store your object in a file named "room.rad".

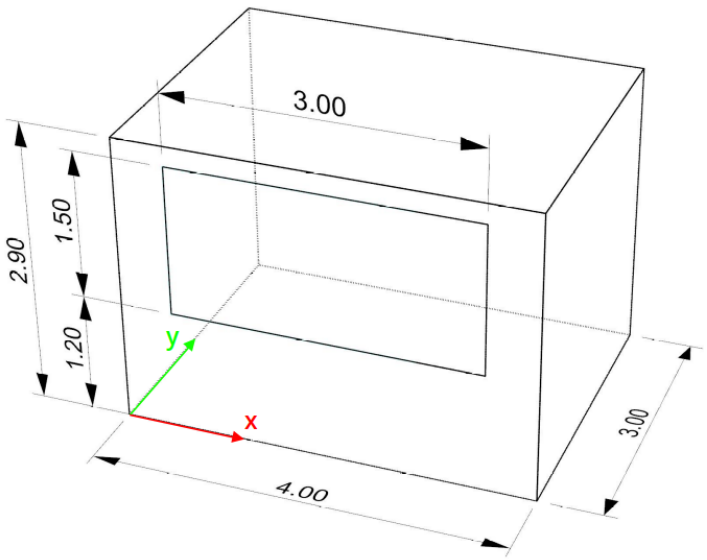


Figure 10: Model of the room used within this section.

2. Assign to the floor, ceiling and walls a `plastic` material with a reflectance in the

range $0.2 - 0.4$, $0.7 - 0.9$ and $0.5 - 0.8$ respectively. The floor material should also be applied the `wooden_pattern` pattern in "materials.mat". Specify your different materials in the later file. How would the surface reflectances affect the incoming light in the space?

3. Using the `polygon` primitive, create a rectangular surface the size of the hole in the window wall. Its position in space must correspond to the exact location of the hole and make sure its normal faces the interior of the space. Store your geometry in a dedicated file named "window.rad" and apply it the material `double_glazing` present in the file "materials.mat".

4. Call `gensky` to generate a description of the sun and sky distributions. Which time and site location have you chosen, and why? Store your data in a file called "sky.mat". The actual sun and ground are already defined in "oustide.rad".

5. Compile your geometries, materials and sky with `oconv`. Store the result in a dedicated file named "scene.oct". Use `rvu` to visualise the scene and make adjustements if required. Employ here the view "inclined.vf" in the "views" folder.

6. When you are happy with your rendering, generate both a luminance and illuminance HDR image of it using the `rpict` command. Use here the view file "inside.vf" and the following parameters for `rpict`:

```
-x 2048 -y 1468 -ab 2 -aa .15 -ar 128 -ad 512 -as 256
```

If required, adjust the exposure in your images using `rpict`, but make sure to save them with a different name.

7. Generate a false colour luminance map of the scene rendered with the view "inside.vf". Adjust the scale accordingly to the range of luminance values in the scene.

8. Similarly, create both illuminance and luminance contour lines and overlay the results onto the luminance HDR image obtained with `rpict`. Compare the two images and interpret the observations.

5 PR - Daylight simulation in a room

It is now time for you to stand on your own two feet. You are given 2 sessions (6h) to produce a lighting rendering from scratch, on the basis of simple guidelines, and subsequently analyse it. The different steps undertaken throughout the project would have to be detailed and commented in a written report. This report should also contain the answers to the different questions hereinafter and any comments you reckon relevant. You may also decide to include visualisations in it directly. At the end of the second session, please make sure to send me all your **Radiance files and renderings** together with your **report**. You can use here again the internal Filesender program from the university and send everything to my email address.

In this last section, you are asked to simulate the lighting within a simple room in RADIANCE, building on the materials we have covered so far. Here are some guidelines that you must follow during the project :

- Your room can be of any reasonable dimensions but must fit in a cube of size 5. Its south-west corner must have coordinates (0, 0, 0) and its floor's main dimension must be parallel to the west-east axis.
- It must be located in France and the exact location is to be stated.
- At the time of the rendering, the sky is sunny and your watch display 14h18.
- Materials for the room surfaces must be carefully selected to meet standard recommendations and the floor must have a dusty wooden appearance.
- One façade of the room must contain a rectangular window with a total transmittance $T_n = 0.76$. Specify the exact size of the window. What transmissivity did you use for your glazing material in RADIANCE?
- A cone with a glossy surface and of basis radius 0.23 must be located inside the space. It can be of any colour and its dimensions must not exceed 0.5 in each direction. What are the coordinates of its centre?
- A second object of your choice must be placed next to the cone, whith a distance of 1.2 between their respective centres. It must have a metallic surface with a blueish hue. Here again, your object should be able to fit in a cube of size 0.5. On which parameters did you act to specify the hue of your object?
- Two light bulbs must be placed along the longest middle axis of the room, just below the ceiling. The distance between their centres must be within the range $1.2 - 1.9$. Specify your value.

All the necessary material for the project is provided in the folder "PR". After compiling your octree scene, generate a high-resolution image of it and use the appropriate commands to analyse the luminance and illuminance distributions inside your indoor space. Discuss the results.

References

[1] Canonical. Ubuntu. URL `https://ubuntu.com`.

[2] A. Jacobs. *UNIX for Radiance Tutorial*. 2010. URL `http://www.jaloxa.eu/resources/radiance/documentation`.

[3] A. Jacobs. *Radiance Tutorial*. 2012. URL `http://www.jaloxa.eu/resources/radiance/documentation`.

[4] J. T. Kajiya. The rendering equation. *SIGGRAPH Comput. Graph.*, 20(4), 1986. doi: 10.1145/15886.15902.

[5] Ladybug Tools. Honeybee. URL `https://www.ladybug.tools/honeybee.html`.

[6] Lawrence Berkeley National Laboratory. The RADIANCE 5.1 Synthetic Imaging System, undated. URL https://floyd.lbl.gov/radiance/refer/ray.html. Accessed: 2023-01-21.

[7] K. Matthews. Radiance Material Notes, 1995. URL http://www.artifice.com/radiance/rad_materials.html. Accessed: 2023-01-21.

[8] A. McNeil. Manual Pages, 2020. URL https://www.radiance-online.org/learning/documentation/manual-pages. Accessed: 2023-01-20.

[9] Solemma. ClimateStudio. URL https://www.solemma.com/climatestudio.

[10] G. L. Ward. Radiance. URL https://www.radiance-online.org.

Appendices

A Main Radiance primitives

Table 2: Main RADIANCE primitives for objects definitions.

Class	Name	Use
Geometry	`sphere`	Define a sphere
	`polygon`	Define a polygon
	`cone`	Define a cone
	`cylinder`	Define a cylinder
	`ring`	Define a ring
	`mesh`	Define a mesh surface, *i.e.* built from multiple triangles
	`source`	Define a distant light source
Materials	`light`	Used for basic self-luminous surfaces, *i.e.* light sources
	`illum`	Used for secondary light sources with a broad distribution (*e.g.* windows)
	`glow`	Used for self-luminous surfaces, but with a limited effect
	`spotlight`	Used for self-luminous surfaces with a directed output
	`mirror`	Used for mirror-like surfaces, *i.e.* highly specular surfaces
	`plastic`	Used for plastic-like surfaces, *i.e.* with uncolored highlights
	`metal`	Used for plastic-like surfaces whose specular components are modified by the material colour
	`trans`	Used for translucent materials
	`glass`	Used for thin glass surfaces and glazings
Textures	`texfunc`	Specify a texture
Patterns	`colorfunc`	Define a colour pattern
	`brightfunc`	Define a monochromatic colour pattern

The complete list of primitives and a their corresponding definitions can be found online at [6].